University of Nottingham
UK | CHINA | MALAYSIA

COMP2054 Tutorial Session 7: Hash Maps and Change Giving

Warren Jackson
Ian Knight
Cas Widdershoven

Session outcomes

- Understand how hash maps work and how to deal with collisions.
- Understand what dynamic programming is and how it can be used to solve (min-coins) change giving problem.

University of Nottingham
UK | CHINA | MALAYSIA

Hash Maps

Array-based Hash Maps with Linear Probing

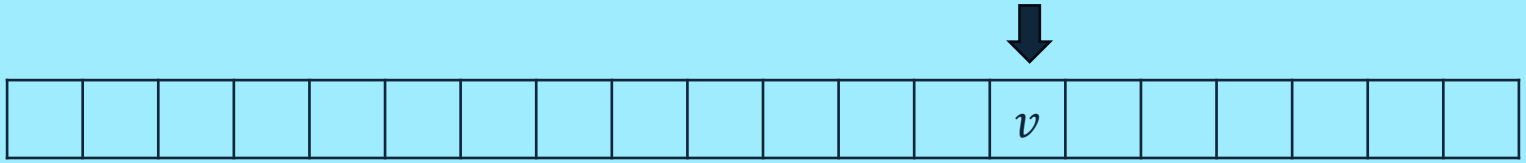
Hash maps

Hashing function to improve average case time complexity of operations from $O(\log n)$ for BST to $O(1)$.

Both BST and hash maps have a worst case of $O(n)$.

Worst case of BST can be improved with self-balancing (e.g. see https://www.geeksforgeeks.org/self-balancing-binary-search-trees/)

$$(k, v) \rightarrow h(k) \rightarrow index$$



A good hash function reduces collisions on the input keys (dispersal)…

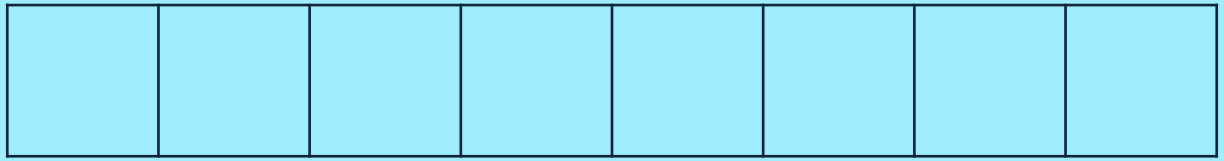
…but is unavoidable so need a mechanism to handle this.

Hash maps – Collision Handling

Can use separate chaining or a method of **open addressing** such as linear probing (today's focus).

Assuming a hash function $h(x) = x \,\%\, 8$,

Insert the following values (in order): $\{2,4,8,16,32,64\}$



Hash maps – Collision Handling

Can use separate chaining or a method of **open addressing** such as linear probing (today's focus).

Assuming a hash function $h(x) = x \,\%\, n$ where $n = 8$.

Insert the following values (in order): $\{2,4,8,16,32,64\}$

Mapping h over the keys gives { 2, 4, 0, 0, 0, 0 }

8	16	2	32	4	64		

We can insert the keys { 2, 4, 8 } into the array at indices { 2, 4, 0 } respectively with no issues.

The problem now comes from inserting the remaining keys which are all mapped to [0] which is occupied.

- With linear probing, we start at [0] and scan right (looping around the array) until we find an empty position where we insert the value; hence, 16 goes into [1], 32 into [3], and 64 into [5].

Hash maps – Collision Handling

Can use separate chaining or a method of **open addressing** such as linear probing (today's focus).
Assuming a hash function $h(x) = x \,\%\, 8$,
Insert the following values (in order): $\{2,4,8,16,32,64\}$

8	16	2	32	4	64		

To do the **get(k)** with linear probing, need to continue scanning until the key is found, an empty cell is found, or we have inspected all cells.

Hash maps – Collision Handling

Can use separate chaining or a method of **open addressing** such as linear probing (today's focus).

Assuming a hash function $h(x) = x \,\%\, n$ where $n = 8$.

Insert the following values (in order): $\{2,4,8,16,32,64\}$

To do the get(k) with linear probing, need to continue scanning until the key is found, an empty cell is found, or we have inspected all cells.

E.g. to `get(2)` we hash 2 to get 2 and look into arr[2]. We find our key and return the value.

To `get(32)` we hash 32 to get 0 and look into arr[0]. We do not find our key so scan right until we find an empty position or the key we are looking for.

Hash maps – Collision Handling

Can use separate chaining or a method of **open addressing** such as linear probing (today's focus).

Assuming a hash function $h(x) = x \,\%\, 8$,

Insert the following values (in order): $\{2,4,8,16,32,64\}$

8	16	2	32	4	64		

Need to handle **remove(k)** using either lazy deletion or reinsertion.

Why is "just removing" a key incorrect?

Hash maps – Collision Handling

Can use separate chaining or a method of **open addressing** such as linear probing (today's focus).

Assuming a hash function $h(x) = x \,\%\, 8$,

Insert the following values (in order): $\{2,4,8,16,32,64\}$

8	16	2	32	4	64		

Need to handle remove(k) using either lazy deletion or reinsertion.

Why is "just removing" a key incorrect?

Here if we were to just remove(32) by setting [3] back to "blank" than we could lose the 64 when performing linear probing. We would start at [0], scan right, and terminate at [3] despite 64 being in [5]. It is therefore necessary to use lazy deletion, marking deleted keys as `-1` or `D` to signal to the probing scheme "continue searching" or by reinserting everything to the right of [3] up until the next blank cell (reinsertion).

Questions: Hash Maps

Question 1:

a. Insert the following keys into a hash map using linear probing with the hash function $h(k) = (k + 1)\ \%\ 7$ into an array of length 7: $\{4, 5, 11\}$.

b. Next remove 4 using an appropriate removal scheme.

c. Then explain how you find the key 11.

Take home exercise (answers to be released on Moodle):

Using the below hash map and hash function, remove each of $\{2,8,16,32\}$ using lazy deletion, then again with reinsertion. Which method required the **least comparisons to find each key** and which method required the **least comparisons overall** (including comparison for the reinsertion)?

$h(x) = x\ \%\ n$ where $n = 8$

8	16	2	32				

Answers: Question 1a

a. Insert the following keys into a hash map using linear probing with the hash function $h(k) = (k + 1)\ \%\ 7$ into an array of length 7: $\{4, 5, 11\}$.

$h(\{4, 5, 11\}) \mapsto \{5, 6, 5\}$

Can insert 4 and 5 into [5] and [6] with no collisions:

[-, -, -, -, -, 4, 5]

Inserting 11 at [5] results in a collision. Scanning right, [6] already contains ‘4’, and so wrap around and find [0] is empty hence can place 11 there.

[11, -, -, -, -, 4, 5]

Answers: Question 1b and 1c

b. Next remove 4 using an appropriate removal scheme.

c. Then explain how you find the key 11.

Current array (hash map): [11, -, -, -, -, 4, 5]

To remove ‘4’ we can mark [5] as deleted or reinsert ‘5’ and ‘11’. The schemes result in [11, -, -, -, -, “D”, 5] or [-, -, -, -, -, 11, 5] respectively.

To find the key 11 with lazy deletion, we inspect [5] and find it is marked as deleted, so try [6] but $5 \neq 11$, next try [(6+1)%7] ([0]) which is 11 so return the associated value.

To find the key 11 with reinsertion, we inspect [5] and find 11 so return the associated value.

Answers: Take home exercise

- *"Using the below hash map and hash function, remove each of* $\{2,8,16,32\}$ *using lazy deletion, then again with reinsertion. Which method required the* ***least comparisons to find each key*** *and which method required the least comparisons overall (including comparison for the reinsertion)?"*

$h(x) = x \,\%\, n$ where $n = 8$

8	16	2	32				

- To complete this exercise, you should do both removeAll with lazy deletion and again with reinsertion with the keys {2, 8, 16, 32} in that order.

	Lazy Deletion	Reinsertion
Find comparisons	8	4
Overall comparisons	8	19

University of Nottingham
UK | CHINA | MALAYSIA

Dynamic Programming

(Min-Coins) Change Giving

Dynamic Programming

A technique used to solve a larger problem by solving smaller "decomposed" subproblems and building-up to the original problem.

Example:

Subset Sum (decision problem) – given a multiset of n integers, $x[0]$ ... $x[n-1]$, is there a subset that sums to a given value, K?

- Smallest subproblem has a multiset of cardinality zero.
- Build up to the original problem by adding the next element from the multiset until all elements are added (or the target value is confirmed).

Subset Sum Example

Example – Subset Sum: given a multiset of integers, $x[0]$ ... $x[n-1]$, is there a subset that sums to a given value, K?

- Smallest subproblem has a multiset of cardinality zero.
- Build up to the original problem by adding the next element from the multiset until all elements are added (or the target value is confirmed).

Given $X = \{1,2,2,5\}$ and $K = 4$:

Value/index	0	1	2	3	4
Possible?	0	0	0	0	0

Subset Sum Example

Example – Subset Sum: given a multiset of integers, $x[0]$ ... $x[n-1]$, is there a subset that sums to a given value, K?

- Smallest subproblem has a multiset of cardinality zero.
- Build up to the original problem by adding the next element from the multiset until all elements are added (or the target value is confirmed).

Given $X = \{1,2,2,5\}$ and $K = 4$: **Considering {}**

Value/index	0	1	2	3	4
Possible?	0	0	0	0	0

Value/index	0	1	2	3	4
Possible?	1	0	0	0	0

Subset Sum Example

Given $X = \{1, 2, 2, 5\}$ and $K = 4$:

Value/index	0	1	2	3	4
Possible?	1	0	0	0	0

Considering {1}

Value/index	0	1	2	3	4
Possible?	1	1	0	0	0

Subset Sum Example

Given $X = \{1, \mathbf{2}, 2,5\}$ and $K = 4$:

Value/index	0	1	2	3	4
Possible?	1	0	0	0	0

Value/index	0	1	2	3	4
Possible?	1	1	0	0	0

Considering {1,2}

Value/index	0	1	2	3	4
Possible?	1	1	1	1	0

Subset Sum Example

Given $X = \{1,2,\mathbf{2},5\}$ and $K = 4$:

Value/index	0	1	2	3	4
Possible?	1	0	0	0	0

Value/index	0	1	2	3	4
Possible?	1	1	0	0	0

Value/index	0	1	2	3	4
Possible?	1	1	1	1	0

Value/index	0	1	2	3	4
Possible?	1	1	1	1	1

Considering {1,2,2}

2 + x[2] == 4; return success;

DP for Min-Coins-Change-Giving

Given a multiset of coins and a target value, K, find the subset of coins which sums to K **with the smallest cardinality**.

```
input: x[0], …, x[n-1] and K
init: Y[0] = 0, Y[m] = -1 (∀m > 0)
for(i=0; i<n; i++) {
  for(m=K-x[i]; m>=0; m--) {
    if(Y[m] >= 0) {
      if(Y[m+x[i]] == -1) {
        Y[m+x[i]] = Y[m]+1
      } else {
        Y[m+x[i]] = min(Y[m+x[i]], Y[m]+1)
      }
    }
  }
}
```

DP for Min-Coins-Change-Giving

Given a multiset of coins and a target value, K, find the subset of coins which sums to K **with the smallest cardinality**.

Basic idea:

- For each element in multiset.
- Scan backwards over an array Y which tells us whether each value was possible using some number of coins for the subsets already considered.
- Introducing a coin of value $x[i]$ means we can give $m + x[i]$ change using $\min(Y[m + x[i]], Y[m] + 1)$ coins.

```
input: x[0], …, x[n-1] and K
init: Y[0] = 0, Y[m] = -1 (∀m > 0)
for(i=0; i<n; i++) {
  for(m=K-x[i]; m>=0; m--) {
    if(Y[m] >= 0) {
      if(Y[m+x[i]] == -1) {
        Y[m+x[i]] = Y[m]+1
      } else {
        Y[m+x[i]] = min(Y[m+x[i]], Y[m]+1)
      }
    }
  }
}
```

DP for Min-Coins-Change-Giving

- Given $X = \{\mathbf{2}, 3,5\}$ and $K = 5$:
- Initialise $Y = [0, -1, -1, -1, -1]$
- First iteration ($\boldsymbol{i = 0}$):

Do nothing for m=3 to m=1 as Y[m]<0

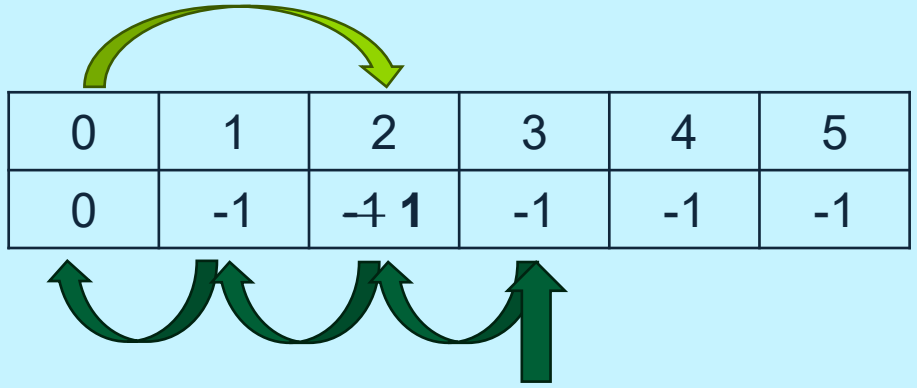
0	1	2	3	4	5
0	-1	-1	-1	-1	-1

m=3

```
input: x[0], …, x[n-1] and K
init: Y[0] = 0, Y[m] = -1 (∀m > 0)
for(i=0; i<n; i++) {
  for(m=K-x[i]; m>=0; m--) {
    if(Y[m] >= 0) {
      if(Y[m+x[i]] == -1) {
        Y[m+x[i]] = Y[m]+1
      } else {
        Y[m+x[i]] = min(Y[m+x[i]], Y[m]+1)
      }
    }
  }
}
```

DP for Min-Coins-Change-Giving

- Given $X = \{\mathbf{2}, 3, 5\}$ and $K = 5$:
- Initialise $Y = [0, -1, -1, -1, -1]$
- First iteration ($\boldsymbol{i = 0}$):
  - Y[0+2] = Y[2] = -1
  - So set Y[0+2] = Y[0]+1 = 0+1 = 1
  - I.e. a sum of 2 can be achieved with 1 coin



0	1	2	3	4	5
0	-1	~~-1~~ **1**	-1	-1	-1

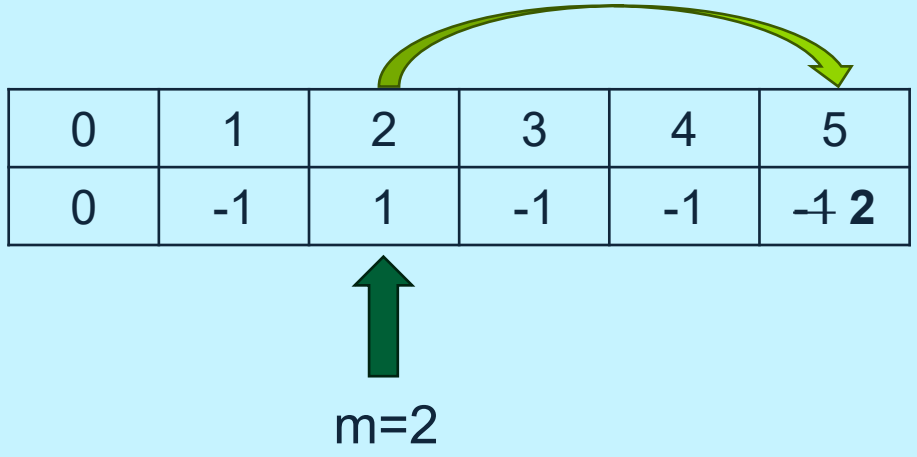
m=0

```
input: x[0], …, x[n-1] and K
init: Y[0] = 0, Y[m] = -1 (∀m > 0)
for(i=0; i<n; i++) {
  for(m=K-x[i]; m>=0; m--) {
    if(Y[m] >= 0) {
      if(Y[m+x[i]] == -1) {
        Y[m+x[i]] = Y[m]+1
      } else {
        Y[m+x[i]] = min(Y[m+x[i]], Y[m]+1)
      }
    }
  }
}
```

DP for Min-Coins-Change-Giving

- Given $X = \{2, \mathbf{3}, 5\}$ and $K = 5$:
- Second iteration ($\boldsymbol{i = 1}$):

  - Y[2+3] = Y[5] = -1
  - So set Y[2+3] = Y[2]+1 = 1+1 = 2
  - I.e. a sum of 5 can be achieved with 2 coins



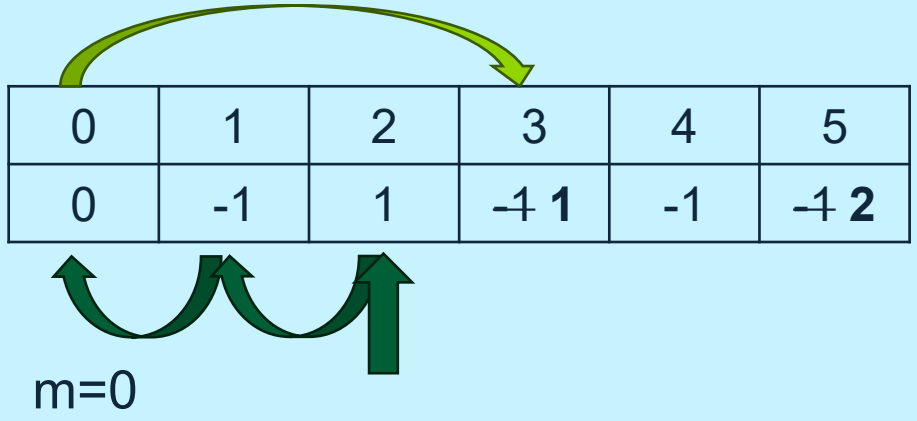
We found that we **can** make change of value 5 using 2 coins, but can we stop here?

```
input: x[0], …, x[n-1] and K
init: Y[0] = 0, Y[m] = -1 (∀m > 0)
for(i=0; i<n; i++) {
  for(m=K-x[i]; m>=0; m--) {
    if(Y[m] >= 0) {
      if(Y[m+x[i]] == -1) {
        Y[m+x[i]] = Y[m]+1
      } else {
        Y[m+x[i]] = min(Y[m+x[i]], Y[m]+1)
      }
    }
  }
}
```

DP for Min-Coins-Change-Giving

- Given $X = \{2, \mathbf{3}, 5\}$ and $K = 5$:
- Second iteration ($\boldsymbol{i = 1}$):

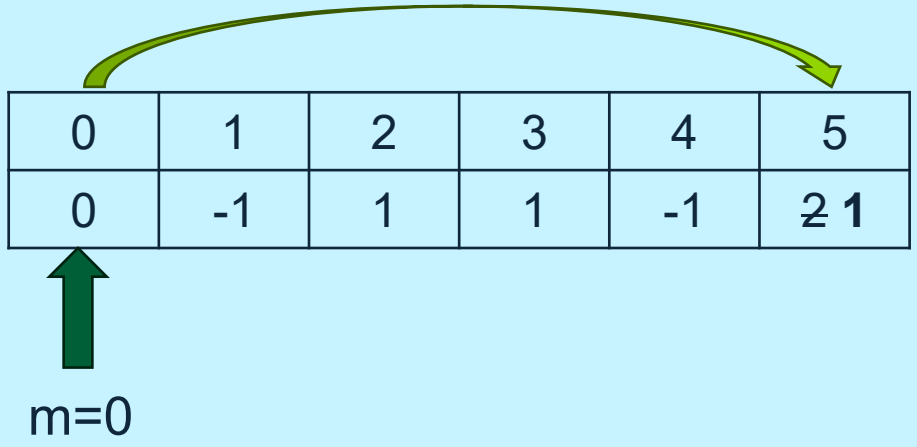
  - Y[0+3] = Y[3] = -1
  - So set Y[0+3] = Y[0]+1 = 0+1 = 1
  - I.e. a sum of 3 can be achieved with 1 coin




```
input: x[0], …, x[n-1] and K
init: Y[0] = 0, Y[m] = -1 (∀m > 0)
for(i=0; i<n; i++) {
  for(m=K-x[i]; m>=0; m--) {
    if(Y[m] >= 0) {
      if(Y[m+x[i]] == -1) {
        Y[m+x[i]] = Y[m]+1
      } else {
        Y[m+x[i]] = min(Y[m+x[i]], Y[m]+1)
      }
    }
  }
}
```

DP for Min-Coins-Change-Giving

- Given $X = \{2,3,\mathbf{5}\}$ and $K = 5$:
- Second iteration ($\boldsymbol{i = 2}$):
  - Y[0+5] = Y[5] = 2
  - So set Y[0+5] = min(Y[0+5],Y[0]+1) = min(2,1) = 1
  - I.e. a sum of 5 can be achieved with 1 coin




```
input: x[0], …, x[n-1] and K
init: Y[0] = 0, Y[m] = -1 (∀m > 0)
for(i=0; i<n; i++) {
  for(m=K-x[i]; m>=0; m--) {
    if(Y[m] >= 0) {
      if(Y[m+x[i]] == -1) {
        Y[m+x[i]] = Y[m]+1
      } else {
        Y[m+x[i]] = min(Y[m+x[i]], Y[m]+1)
      }
    }
  }
}
```

Questions

Q1. Given the set of coins $\{1,1,3,5,9\}$, what is the minimum number of coins required to give change of 10?

Q2. Given the set of coins $\{2,2,2,5\}$, what is the minimum number of coins required to give change of 6?

Q3. Does the DP algorithm still work if we scan the array, Y, forwards instead of backwards? If yes, explain; if no, give a counter example.

Answers

Q1. Given the set of coins $\{1,1,3,5,9\}$, what is the minimum number of coins required to give change of 10?

K = 10 can be achieved with 2 coins.

Initialise Y:

0	1	2	3	4	5	6	7	8	9	10
0	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

i=0, x[i]=1:

0	1	2	3	4	5	6	7	8	9	10
0	**1**	-1	-1	-1	-1	-1	-1	-1	-1	-1

i=1, x[i]=1:

0	1	2	3	4	5	6	7	8	9	10
0	**1**	**2**	-1	-1	-1	-1	-1	-1	-1	-1

i=2, x[i]=3:

0	1	2	3	4	5	6	7	8	9	10
0	1	2	**1**	**2**	**3**	-1	-1	-1	-1	-1

i=3, x[i]=5:

0	1	2	3	4	5	6	7	8	9	10
0	1	2	1	2	**1**	**2**	**3**	**2**	**3**	**4**

i=4, x[i]=9:

0	1	2	3	4	5	6	7	8	9	10
0	1	2	1	2	1	2	3	2	**1**	**2**

Answers

Q2. Given the set of coins $\{2,2,2,5\}$, what is the minimum number of coins required to give change of 6?

Initialise Y:

0	1	2	3	4	5	6
0	-1	-1	-1	-1	-1	-1

i=0, x[i]=2:

0	1	2	3	4	5	6
0	-1	**1**	-1	-1	-1	-1

i=1, x[i]=2:

0	1	2	3	4	5	6
0	-1	**1**	-1	**2**	-1	-1

i=2, x[i]=2:

0	1	2	3	4	5	6
0	-1	**1**	-1	**2**	-1	**3**

i=3, x[i]=5:

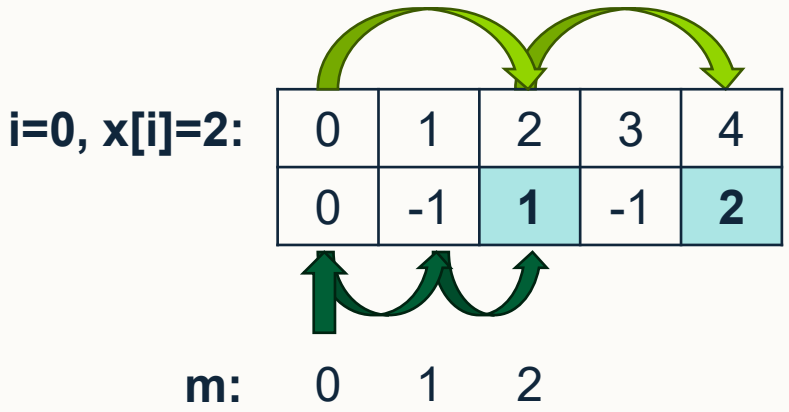
0	1	2	3	4	5	6
0	-1	1	-1	2	**1**	3

K = 6 can be achieved with 3 coins.

Answers

Q3. Does the DP algorithm still work if we scan the array, Y, forwards instead of backwards? If yes, explain; if no, give a counter example.

Using the inputs: {2,1,1} and K=4. If we scan Y forwards (i.e. we start the loop variable m at 0 and increment it at every step until we reach K-x[i]), then we would get the following first iteration:



We should only be considering a single coin in the first iteration, but Y has been updated to supposedly show that 4 is achievable with two coins. However, we can see from our input coins {2,1,1} that we should only be able to achieve a sum of 4 **by using all coins.**

Consequently, the DP algorithm is not guaranteed to give the correct answer for all problems if we can Y forwards.

University of Nottingham
UK | CHINA | MALAYSIA

Thank you